

Index

1. Resources
2. Problem statement and graph
3. PDDL domain using STRIPS
4. Plasp translation
5. Use of clingo ed eclingo
 - a. Screen 1 e 2 for clingo
 - b. Screen from 3 to 9 for eclingo
6. ASP(Q) file from plasp output
 - a. qasp.jar screenshots
7. K language
 - a. DLV commands
 - b. Xml style file
 - c. Reverse plan library

1) Resources

I needed to use a variety of resources to evaluate the created planning domain.

Plasp : to translate my PDDL files in ASP ones.

Clingo : grounder and solver

Eclingo : epistemic clingo (let you use epistemic operators inside code)

DLV^k : is builded on top of DLV, we use this with k language files

2) Problem statement and graph

The domain I chose to create concerns the steps necessary to create a Sicilian cannolo, since Sicily is my homeland.

Specifically, I have created a simpler domain and a slightly more complex one.

The basic idea, however, is that it is possible to take a cannolo, throw it, fry it and finally fill it.

In the hard case you also have a fluent "sacpoche" which indicates whether you have put the cotta cheese in the sacpoche, so you can fill the cannolo.

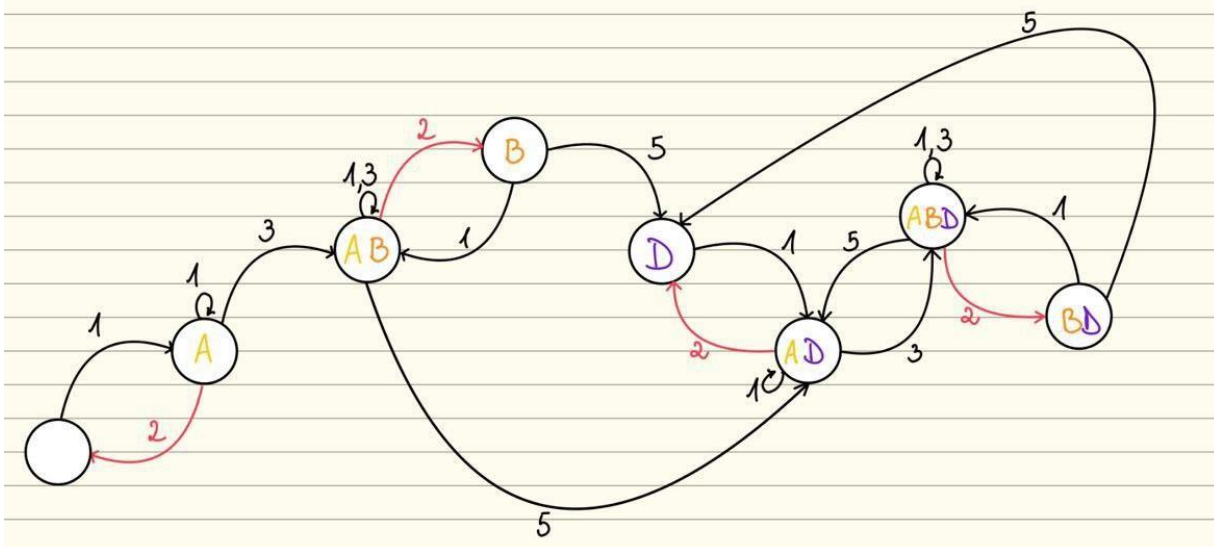
In the simple case, on the other hand, it is filled directly (without the aid of the sacpoche).

On the following 2 pages, you can see the graph corresponding to the .pddl domain to be created.

Simple domain)

Fluents		Actions	
- taken_cannolo	A	- take_cannolo	1
- fried_cannolo	B	- discard_cannolo	2
- stuffed_cannolo	D	- fry_cannolo	3
		- stuff_cannolo	5

Action	Identifier	Precondition	Positive effect	Negative effect
take_cannolo	1		taken_cannolo A	
discard_cannolo	2	taken_cannolo A		taken_cannolo A
fry_cannolo	3	taken_cannolo A	fried_cannolo B	
stuff_cannolo	5	fried_cannolo B	stuffed_cannolo D	fried_cannolo B



This is the state transition system in which we represent states as nodes and actions as arcs.

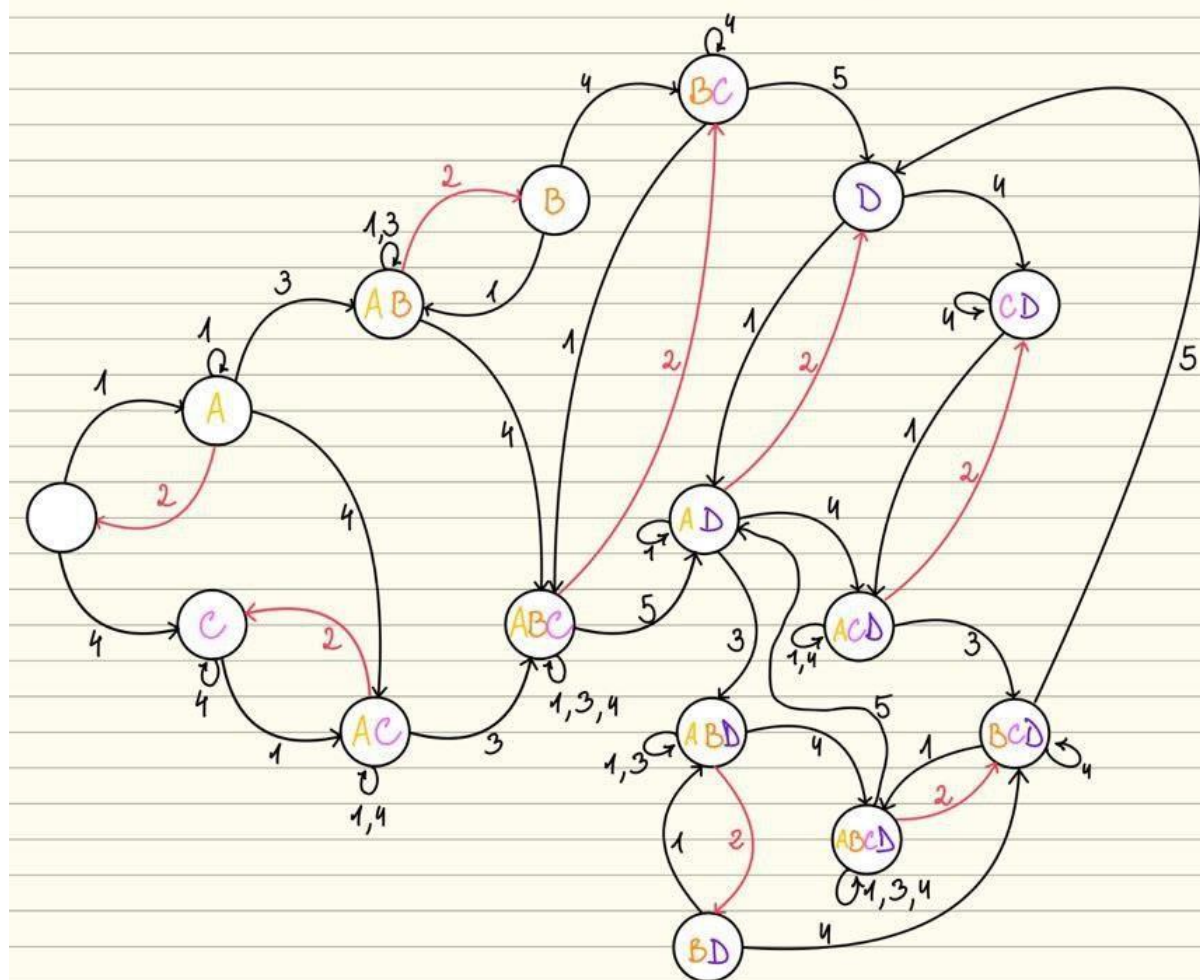
The arcs in red highlight the application of action 2, which is uniformly reversible because whenever it is applicable, i.e. when “taken_cannolo” (A) is true, action 1 can return us to the **original state**.

This property is true for all states, so action ‘discard_cannulus’ (2) is **universally uniformly reversible**.

Hard domain)

Fluents		Actions	
- taken_cannolo	A	- take_cannolo	1
- fried_cannolo	B	- discard_cannolo	2
- sacpoche_filled	C	- fry_cannolo	3
- stuffed_cannolo	D	- add_cottage_cheese	4
		- stuff_cannolo	5

Action	Identifier	Precondition	Positive effect	Negative effect
take_cannolo	1		taken_cannolo A	
discard_cannolo	2	taken_cannolo A		taken_cannolo A
fry_cannolo	3	taken_cannolo A	fried_cannolo B	
add_cottage_cheese	4		sacpoché_filled C	
stuff_cannolo	5	fried_cannolo B sacpoché_filled C	stuffed_cannolo D	fried_cannolo B sacpoché_filled C



3) PDDL domain using STRIP

I therefore created the two domains `simple_cannolo_siciliano` and `hard_cannolo_siciliano`, respectively, in the files:

- `cannolo_simple.pddl`
- `cannolo_hard.pddl`

The actions that can be used in this domain do not use parameters, on the contrary they sometimes require preconditions (only present in a positive form) and effects, which can be positive in the sense that they make a fluent true, or negative in the sense that they make a fluent false.

4) Plasp translation

I then used **plasp** to perform the translation from **PDDL** to **ASP**.

In doing so, I obtained 2 files:

- `cannolo_plasp.lp`
- `cannolo_plasp_hard.lp`

5) Use of clingo and eclingo

Proceed now to run the solver on the output produced by **plasp**.

a) clingo screenshots

In the 'Screen' folder, you can view the screens presented in this section 5).

- Screen 1 : `clingo .\sequential-horizon.uurev.sat.lp cannolo_plasp.lp`

```
(potassco) D:\Specialistica\Advanced Topics in Ai 2\5 - Reversability\reversability-asp-elp-aspq\cartella_esperimenti>clingo cannolo\cannolo_plasp.lp ..\sequential-horizon.uurev.sat.lp
clingo version 5.7.1
Reading from cannolo\cannolo_plasp.lp ...
Solving...
Answer: 1
chosen("discard_cannolo") plan("take_cannolo",1)
SATISFIABLE

Models      : 1+
Calls       : 1
Time        : 0.007s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.016s
```

- Screen 2 : `clingo .\sequential-horizon.uurev.sat.lp cannolo_plasp_hard.lp`

```
(potassco) D:\Specialistica\Advanced Topics in Ai 2\5 - Reversability\reversability-asp-elp-aspq\cartella_esperimenti>clingo ..\sequential-horizon.uurev.sat.lp ..\cannolo\cannolo_plasp_hard.lp
clingo version 5.7.1
Reading from ..\sequential-horizon.uurev.sat.lp ...
Solving...
Answer: 1
chosen("discard_cannolo") plan("take_cannolo",1)
SATISFIABLE

Models      : 1+
Calls       : 1
Time        : 0.022s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time    : 0.000s
```

b) eclingo screenshots

The idea is to execute a command like this:

eclingo 0 .\sequential-horizon.uurev.eclingo cannolo_plasp.lp -c horizon= X

- Screen 3 : cannolo_plasp.lp and horizon = 1

```
(eclingo) D:\Specialistica\Advanced Topics in AI 2\5 - Reversability\reversibility-asp-elp-aspq\cartella_esperimenti>eclingo 0 .\sequential-horizon.uurev.eclingo .\cannolo\cannolo_plasp.lp -c horizon=1
Solving...
World view: 1
&k(chosen("discard_cannolo")) &k(plan("take_cannolo",1))
SATISFIABLE
```

- Screen 4 : cannolo_plasp.lp and horizon = 2

```
(eclingo) D:\Specialistica\Advanced Topics in AI 2\5 - Reversability\reversibility-asp-elp-aspq\cartella_esperimenti>eclingo 0 .\sequential-horizon.uurev.eclingo .\cannolo\cannolo_plasp.lp -c horizon=2
Solving...
World view: 1
&k(chosen("discard_cannolo")) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",2))
SATISFIABLE
```

- Screen 5 : cannolo_plasp.lp , horizon = 1 and .general.eclingo

```
(eclingo) D:\Specialistica\Advanced Topics in AI 2\5 - Reversability\reversibility-asp-elp-aspq\cartella_esperimenti>eclingo 0 .\sequential-horizon.uurev.general.eclingo .\cannolo\cannolo_plasp.lp -c horizon=1
Solving...
World view: 1
&k(chosen("discard_cannolo")) &k(plan("take_cannolo",1))
SATISFIABLE
```

- Screen 6 : cannolo_plasp_hard.lp and horizon = 1

```
(potassco) D:\Specialistica\Advanced Topics in AI 2\5 - Reversability\reversibility-asp-elp-aspq\cartella_esperimenti>conda activate eclingo
(eclingo) D:\Specialistica\Advanced Topics in AI 2\5 - Reversability\reversibility-asp-elp-aspq\cartella_esperimenti>eclingo 0 .\sequential-horizon.uurev.eclingo .\cannolo\cannolo_plasp_hard.lp -c horizon=1
Solving...
World view: 1
&k(chosen("discard_cannolo")) &k(plan("take_cannolo",1))
SATISFIABLE
```

- Screen 7 : cannolo_plasp_hard.lp and horizon = 2

```
(eclingo) D:\Specialistica\Advanced Topics in AI 2\5 - Reversability\reversibility-asp-elp-aspq\cartella_esperimenti>eclingo 0 .\sequential-horizon.uurev.eclingo .\cannolo\cannolo_plasp_hard.lp -c horizon=2
Solving...
World view: 1
&k(chosen("discard_cannolo")) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",2))
SATISFIABLE
```

- Screen 8 : cannolo_plasp_hard.lp and horizon = 6

```
(eclingo) D:\Specialistica\Advanced Topics in AI 2\5 - Reversability\reversibility-asp-elp-aspq\cartella_esperimenti>eclingo 0 .\sequential-horizon.uurev.eclingo .\cannolo\cannolo_plasp_hard.lp -c horizon=6
Solving...
World view: 1
&k(chosen("discard_cannolo")) &k(plan("discard_cannolo",2)) &k(plan("discard_cannolo",4)) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",3)) &k(plan("take_cannolo",5)) &k(plan("take_cannolo",6))
World view: 2
&k(chosen("discard_cannolo")) &k(plan("discard_cannolo",2)) &k(plan("discard_cannolo",5)) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",3)) &k(plan("take_cannolo",4)) &k(plan("take_cannolo",6))
World view: 3
&k(chosen("discard_cannolo")) &k(plan("discard_cannolo",2)) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",3)) &k(plan("take_cannolo",4)) &k(plan("take_cannolo",5)) &k(plan("take_cannolo",6))
World view: 4
&k(chosen("discard_cannolo")) &k(plan("discard_cannolo",3)) &k(plan("discard_cannolo",5)) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",2)) &k(plan("take_cannolo",4)) &k(plan("take_cannolo",6))
World view: 5
&k(chosen("discard_cannolo")) &k(plan("discard_cannolo",5)) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",2)) &k(plan("take_cannolo",3)) &k(plan("take_cannolo",4)) &k(plan("take_cannolo",6))
World view: 6
&k(chosen("discard_cannolo")) &k(plan("discard_cannolo",3)) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",2)) &k(plan("take_cannolo",4)) &k(plan("take_cannolo",5)) &k(plan("take_cannolo",6))
World view: 7
&k(chosen("discard_cannolo")) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",2)) &k(plan("take_cannolo",3)) &k(plan("take_cannolo",4)) &k(plan("take_cannolo",5)) &k(plan("take_cannolo",6))
World view: 8
&k(chosen("discard_cannolo")) &k(plan("discard_cannolo",4)) &k(plan("take_cannolo",1)) &k(plan("take_cannolo",2)) &k(plan("take_cannolo",3)) &k(plan("take_cannolo",5)) &k(plan("take_cannolo",6))
SATISFIABLE
```

- Screen 9 : cannolo_plasp_hard.lp, horizon = 1 and .general.eclingo

```
(eclingo) D:\Specialistica\Advanced Topics in AI 2\5 - Reversability\reversibility-asp-elp-aspq\cartella_esperimenti>eclingo 0 .\sequential-horizon.uurev.general.eclingo .\cannolo\cannolo_plasp_hard.lp -c horizon=1
Solving...
World view: 1
&k(chosen("discard_cannolo")) &k(plan("take_cannolo",1))
SATISFIABLE
```

6) ASP(Q) file from plasp output

The next step is to use the two files cannolo_plasp.lp and cannolo_plasp_hard.lp produced by plasp, i.e. the two domains, and insert them into *program 1* relating to ASP(Q) encoding for **Uniform Reversibility**, i.e. the one related to the **existential operator**.

This *program 1* contains the code that is listed under *@exists*, i.e. there is an initial part and then the domain (which is the content of the output of plasp).

Then I added the code for the *program 2* (relating to the **universally operator**) denoted by what is under *@forall*, and also adding the part relating to *@constraints*.

Note that I refer to these three *@...* for convenience of reference, but the code is actually contained all together in one file.

Doing so, I created the two file:

- cannolo_simple_qasp.lp
- cannolo_hard_qasp.lp

Once this was done, I proceeded to change the file extension, resulting in:

- cannolo_simple_qasp.aspq
- cannolo_hard_qasp.aspq

The next step is to use the provided asp.jar file to execute the files.

The idea is to execute a command like this:

```
java -jar ./qasp.jar -n -1 -m file.aspq
```

a) qasp.jar screenshots

Note that here I needed to ask a colleague of mine to execute this part because I had problem using the qasp.jar file.

These are the outputs of using my aspq files with the provided qasp.jar file.

- Screen 10 : java -jar ./qasp.jar -n -1 -m cannolo_simple_qasp.aspq

```

Cristian@ubuntu:~/Desktop/tautonico_project$ java -jar reversibility-tools-2025/qasp.jar -n -1 -m EDO/QASP_FILES/cannolo_simple_qasp.aspq
SAT
(occurs("discard_cannolo",1), chosen("discard_cannolo"), plan("take_cannolo",1), occurs("take_cannolo",2), time(0), time(1), time(2), action(action("take_cannolo")
), action(action("discard_cannolo")), action(action("fry_cannolo")), action(action("add_cottage_cheese")), action(action("stuff_cannolo")), boolean(true), boolean(
false), variable(variable("taken_cannolo")), variable(variable("fried_cannolo")), variable(variable("sacpoche_filled")), variable(variable("stuffed_cannolo")), con
tains(variable("taken_cannolo"),value(variable("taken_cannolo"),true)), contains(variable("fried_cannolo"),value(variable("fried_cannolo"),true)), contains(variab
le("sacpoche_filled"),value(variable("sacpoche_filled"),true)), contains(variable("stuffed_cannolo"),value(variable("stuffed_cannolo"),true)), contains(variable("ta
ken_cannolo"),value(variable("taken_cannolo"),false)), contains(variable("fried_cannolo"),value(variable("fried_cannolo"),false)), contains(variable("sacpoche_fill
ed"),value(variable("sacpoche_filled"),false)), contains(variable("stuffed_cannolo"),value(variable("stuffed_cannolo"),false)), postcondition(action("stuff_cannolo"),effect(unconditional),variable("stuffed_cannolo"),true)), postcondition(action("add_cottage_cheese"),effect(unconditional),variable("sacpoche_filled"),value(variable("sacpoche_filled"),true)), p
ostcondition(action("fry_cannolo"),effect(unconditional),variable("fried_cannolo"),value(variable("fried_cannolo"),true)), postcondition(action("discard_cannolo"),effect(unconditional),variable("taken_cannolo"),value(variable("taken_cannolo"),false)), postcondition(action("take_cannolo"),effect(unconditional),variable("taken_cannolo"),value(variable("taken_cannolo"),true)), precondition(action("stuff_cannolo"),variable("sacpoche_filled"),value(variable("sacpoche_filled"),true)), precondition(action("fry_cannolo"),variable("taken_cannolo"),value(variable("taken_cannolo"),true)), precondition(action("discard_cannolo"),variable("taken_cannolo"),value(variable("taken_cannolo"),true)), horizon(1), type(type("object"))))

```

- Screen 11 : java -jar ./qasp.jar -n -1 -m cannolo_hard_qasp.aspq

```

Cristian@ubuntu:~/Desktop/tautonico_project$ java -jar reversibility-tools-2025/qasp.jar -n -1 -m EDO/QASP_FILES/cannolo_hard_qasp.aspq
SAT
(occurs("discard_cannolo",1), chosen("discard_cannolo"), plan("take_cannolo",1), occurs("take_cannolo",2), time(0), time(1), time(2), action(action("take_cannolo")
), action(action("discard_cannolo")), action(action("fry_cannolo")), action(action("add_cottage_cheese")), action(action("stuff_cannolo")), boolean(true), boolean(
false), variable(variable("taken_cannolo")), variable(variable("fried_cannolo")), variable(variable("sacpoche_filled")), variable(variable("stuffed_cannolo")), con
tains(variable("taken_cannolo"),value(variable("taken_cannolo"),true)), contains(variable("fried_cannolo"),value(variable("fried_cannolo"),true)), contains(variab
le("sacpoche_filled"),value(variable("sacpoche_filled"),true)), contains(variable("stuffed_cannolo"),value(variable("stuffed_cannolo"),true)), contains(variable("ta
ken_cannolo"),value(variable("taken_cannolo"),false)), contains(variable("fried_cannolo"),value(variable("fried_cannolo"),false)), contains(variable("sacpoche_fill
ed"),value(variable("sacpoche_filled"),false)), contains(variable("stuffed_cannolo"),value(variable("stuffed_cannolo"),false)), postcondition(action("stuff_cannolo"),effect(unconditional),variable("stuffed_cannolo"),true)), postcondition(action("add_cottage_cheese"),effect(unconditional),variable("sacpoche_filled"),value(variable("sacpoche_filled"),true)), p
ostcondition(action("fry_cannolo"),effect(unconditional),variable("fried_cannolo"),value(variable("fried_cannolo"),true)), postcondition(action("discard_cannolo"),effect(unconditional),variable("taken_cannolo"),value(variable("taken_cannolo"),false)), postcondition(action("take_cannolo"),effect(unconditional),variable("taken_cannolo"),value(variable("taken_cannolo"),true)), precondition(action("stuff_cannolo"),variable("sacpoche_filled"),value(variable("sacpoche_filled"),true)), precondition(action("fry_cannolo"),variable("taken_cannolo"),value(variable("taken_cannolo"),true)), precondition(action("discard_cannolo"),variable("taken_cannolo"),value(variable("taken_cannolo"),true)), horizon(1), type(type("object"))))

```

- Screen 12 : java -jar ./qasp.jar -n -1 -m -f chosen, plan cannolo_simple_qasp.aspq

```

Cristian@ubuntu:~/Desktop/tautonico_project$ java -jar reversibility-tools-2025/qasp.jar -n -1 -m -f chosen,plan EDO/QASP_FILES/cannolo_hard_qasp.aspq
SAT
(chosen("discard_cannolo"), plan("take_cannolo",1))

```

- Screen 13 : java -jar ./qasp.jar -n -1 -m -f chosen, plan cannolo_hard_qasp.aspq

```

Cristian@ubuntu:~/Desktop/tautonico_project$ java -jar reversibility-tools-2025/qasp.jar -n -1 -m -f chosen,plan EDO/QASP_FILES/cannolo_simple_qasp.aspq
SAT
(chosen("discard_cannolo"), plan("take_cannolo",1))

```

7) K language

Working about the K language, I created an equivalent version of the domain previously writed in PDDL.

- cannolo_simple_k.plan
- cannolo_hard_k.plan

The **goal** is set for both files to "stuffed_cannolo".

a) DLV commands to execute files .plan

Note that here we use -FP that means Front-end Planning

- Screen 14 : cannolo_simple_k.plan and planlength = 3

```
cristian@ubuntu:~/Desktop/tautonico_project$ reversibility-tools-2025/DLV -FP -planlength=3 EDO/K_files/cannolo_simple_k.plan
DLV [build BEN/Dec 17 2012 gcc 4.6.1]

STATE 0:
ACTIONS: take_cannolo
STATE 1: taken_cannolo
ACTIONS: fry_cannolo
STATE 2: taken_cannolo, fried_cannolo
ACTIONS: stuff_cannolo
STATE 3: taken_cannolo, -fried_cannolo, stuffed_cannolo
PLAN: take_cannolo; fry_cannolo; stuff_cannolo

Check whether that plan is secure (y/n)? y
The plan is secure.

Search for other plans (y/n)? █
```

- Screen 15 : cannolo_hard_k.plan and planlength = 5

```
cristian@ubuntu:~/Desktop/tautonico_project$ reversibility-tools-2025/DLV -FP -planlength=5 EDO/K_files/cannolo_hard_k.plan
DLV [build BEN/Dec 17 2012 gcc 4.6.1]

STATE 0:
ACTIONS: take_cannolo
STATE 1: taken_cannolo
ACTIONS: fry_cannolo
STATE 2: taken_cannolo, fried_cannolo
ACTIONS: add_cottage_cheese
STATE 3: taken_cannolo, sacpoche_filled, fried_cannolo
ACTIONS: take_cannolo
STATE 4: taken_cannolo, sacpoche_filled, fried_cannolo
ACTIONS: stuff_cannolo
STATE 5: taken_cannolo, -sacpoche_filled, -fried_cannolo, stuffed_cannolo
PLAN: take_cannolo; fry_cannolo; add_cottage_cheese; take_cannolo; stuff_cannolo

Check whether that plan is secure (y/n)? y
The plan is secure.

Search for other plans (y/n)?
```

- Screen 16 : cannolo_hard_k.plan and planlength = 4

```
cristian@ubuntu:~/Desktop/tautonico_project$ reversibility-tools-2025/DLV -FP -planlength=4 EDO/K_files/cannolo_hard_k.plan
DLV [build BEN/Dec 17 2012 gcc 4.6.1]

STATE 0:
ACTIONS: take_cannolo
STATE 1: taken_cannolo
ACTIONS: fry_cannolo
STATE 2: taken_cannolo, fried_cannolo
ACTIONS: add_cottage_cheese
STATE 3: taken_cannolo, sacpoche_filled, fried_cannolo
ACTIONS: stuff_cannolo
STATE 4: taken_cannolo, -sacpoche_filled, -fried_cannolo, stuffed_cannolo
PLAN: take_cannolo; fry_cannolo; add_cottage_cheese; stuff_cannolo

Check whether that plan is secure (y/n)? y
The plan is secure.
```

- Screen 17 : cannolo_hard_k.plan, planlength = 4 and -FPopt

Here -FPopt is used to search for optimistic plans.

```
cristian@ubuntu:~/Desktop/tautonic_project$ reversibility-tools-2025/DLV -FPopt -planlength=4 EDO/K_files/cannolo_hard_k.plan
DLV [build BEN/Dec 17 2012 gcc 4.6.1]

clear: line 0: can't open input.
STATE 0:
ACTIONS: take_cannolo
STATE 1: taken_cannolo
ACTIONS: fry_cannolo
STATE 2: taken_cannolo, fried_cannolo
ACTIONS: add_cottage_cheese
STATE 3: taken_cannolo, sacpoche_filled, fried_cannolo
ACTIONS: stuff_cannolo
STATE 4: taken_cannolo, -sacpoche_filled, -fried_cannolo, stuffed_cannolo
PLAN: take_cannolo; fry_cannolo; add_cottage_cheese; stuff_cannolo

STATE 0:
ACTIONS: take_cannolo
STATE 1: taken_cannolo
ACTIONS: add_cottage_cheese
STATE 2: taken_cannolo, sacpoche_filled
ACTIONS: fry_cannolo
STATE 3: taken_cannolo, sacpoche_filled, fried_cannolo
ACTIONS: stuff_cannolo
STATE 4: taken_cannolo, -sacpoche_filled, -fried_cannolo, stuffed_cannolo
PLAN: take_cannolo; add_cottage_cheese; fry_cannolo; stuff_cannolo

STATE 0:
ACTIONS: add_cottage_cheese
STATE 1: sacpoche_filled
ACTIONS: take_cannolo
STATE 2: taken_cannolo, sacpoche_filled
ACTIONS: fry_cannolo
STATE 3: taken_cannolo, sacpoche_filled, fried_cannolo
ACTIONS: stuff_cannolo
STATE 4: taken_cannolo, -sacpoche_filled, -fried_cannolo, stuffed_cannolo
PLAN: add_cottage_cheese; take_cannolo; fry_cannolo; stuff_cannolo
```

b) xml style file

This xml sytle file named *condition_domain.xml* is used to select the desired element that needs to be true.

Note that here I needed to ask again to a colleague of mine to execute this last thing.

This is the reason why the path here hardcoded is like that.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE conditions SYSTEM "/home/cristian/Desktop/tautonic_project/reversibility-tools-2025/RevPlan/bin/DTD/conditions.dtd">
3 <conditions>
4   <phi>
5     <term> true </term>
6   </phi>
7   <psi>
8     <and>
9       <term> taken_cannolo </term>
10    </and>
11  </psi>
12 </conditions>
```

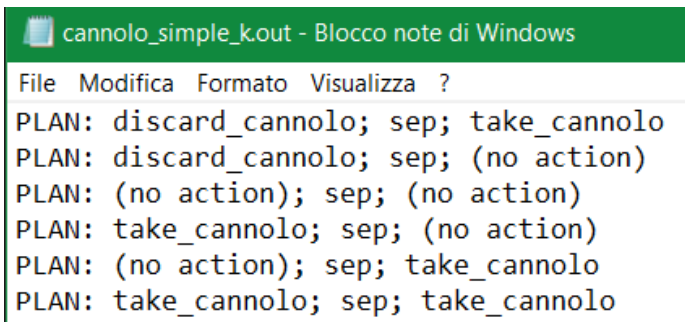
c) Reverse plan library

Lastly obtained the library that has on the left an action, then separator, and then the path (sequence of action) that reverts the action before the separator.

- Screen 18 : obtain reverse plan library in the simple case

```
cristian@ubuntu: ~/Desktop/tautonico_project/reversibility-tools-2025/RevPlan/bin$ java -cp plan-library-binary.jar planlibrary.ReverseDomain -x ../../ED0/K_files/cannolo_simple_k.plan ../../ED0/K_files/background_knowledge.dl ../../ED0/K_files/condition_domain.xml ../../reversibility-tools-2025/DLV 3
Parsing domain: cannolo_simple_k
Finished parsing domain ... starting to compute reverse plans using DLV
./scripts/runDLV.sh ../../reversibility-tools-2025/DLV 3 opt "/home/cristian/Desktop/tautonico_project/reversibility-tools-2025/RevPlan/bin/../../ED0/K_files/background_knowledge.dl" "/home/cristian/Desktop/tautonico_project/reversibility-tools-2025/RevPlan/bin/cannolo_simple_k.sinrev.plan" "/home/cristian/Desktop/tautonico_project/reversibility-tools-2025/RevPlan/bin/cannolo_simple_k.rev.out"
Finished computing reverse plans ... starting to compare sinrev.out and rev.out
Finished comparing output - reverse plans stored in: /home/cristian/Desktop/tautonico_project/reversibility-tools-2025/RevPlan/bin/../../ED0/K_files/cannolo_simple_k.out
```

Content of file in output:



```
File Modifica Formato Visualizza ?
PLAN: discard_cannolo; sep; take_cannolo
PLAN: discard_cannolo; sep; (no action)
PLAN: (no action); sep; (no action)
PLAN: take_cannolo; sep; (no action)
PLAN: (no action); sep; take_cannolo
PLAN: take_cannolo; sep; take_cannolo
```